

MACHINE LEARNING, TOLD SIMPLY

Book 1 · Contact Session 1

# What Does It Mean for a Machine to Learn?

*Introduction to Machine Learning — what it is, when to use it,  
the three families of learning, and the workflow every ML  
project follows.*

---

Machine Learning · AIML ZG565 / DSE ZG565 · BITS Pilani WILP

Companion to Contact Session 1 (Module 1: Introduction)

Companion notes prepared by Prof. Saurabh

## What is inside

This book follows Meera, a new data analyst at a used-car marketplace, as she meets machine learning for the first time. Read it before or after the Session 1 slides — it explains every idea on the slides in plain language, with pictures and everyday examples, and ends with a self-check.

|          |                                              |                                                                                                                           |
|----------|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>1</b> | <b>The Problem You Cannot Code by Hand</b>   | Why some jobs are impossible to program line by line — and what we do instead.                                            |
| <b>2</b> | <b>Task, Performance, Experience</b>         | The one definition of learning you must be able to write in an exam — and how to apply it to any problem.                 |
| <b>3</b> | <b>When Should You Use Machine Learning?</b> | Learning is powerful but not free. Four situations that call for it — and one that does not.                              |
| <b>4</b> | <b>The Three Families of Learning</b>        | Supervised, unsupervised and reinforcement learning — sorted by one question: what kind of feedback does the learner get? |
| <b>5</b> | <b>Two More Ways to Slice Learning</b>       | How much data the learner swallows at a time, and whether it keeps the data or keeps a rule.                              |
| <b>6</b> | <b>The Machine Learning Workflow</b>         | Seven steps that every ML project goes through — walked end to end on the car-price problem.                              |
| <b>7</b> | <b>What Can Go Wrong (a Preview)</b>         | The issues in machine learning that the rest of the course is built to solve.                                             |
| <b>8</b> | <b>Self-Check and the Road Ahead</b>         | Twelve quick questions to test yourself, with answers, and a map of how Session 1 connects to the rest of the course.     |

## How to read this book

Every chapter follows the same rhythm: a short **story** to set the scene, the **big idea** in one line, an everyday **analogy**, then the **maths step by step** — always the plain version first, and the matrix version only after the plain one is clear. Boxes to watch for:

### STORY

Meera at Sahi Deal — the running example.

### BIG IDEA

The one sentence to remember.

### THINK OF IT LIKE

A real-life picture of the concept.

### STOP AND THINK

A question to try before reading on.

### OUR ANSWER

The answer, right after the question.

### CAREFUL

A common mistake and how to avoid it.

### MATHS, STEP BY STEP

Every line of algebra, nothing skipped.

### WORKED EXAMPLE

Real numbers, worked in full.

### EXAM TIP

How this topic is asked, and how to score.

### IN ONE BREATH

The chapter in a few lines.

## CHAPTER 1

# The Problem You Cannot Code by Hand

*Why some jobs are impossible to program line by line — and what we do instead.*

## STORY

Meera's first morning at **Sahi Deal**, a used-car marketplace in Pune. Her manager, Arjun, drops a spreadsheet of 40,000 old listings on her desk. "Every seller asks us the same question: *what is my car worth?* Today we answer by guesswork. I want a program that gives a fair price in one second." Meera opens her editor and starts typing rules: *if the car is a Honda City and it is from 2019 and it has done under 40,000 km then...* Two hours later she has forty rules, and the fifth rule already contradicts the twelfth.

## Two ways to make a computer do something

There are only two ways to get a computer to solve a problem. The first is the way we all learnt: **you write the rules**. You think hard, you write *if this then that*, and the computer follows your rules exactly. This is **traditional programming**. Data goes in, your program runs on it, and an answer comes out.

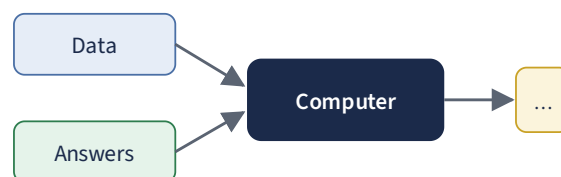
The second way is newer and stranger. **You show the computer examples of the answer**, and let it work out the rules itself. Data goes in *together with* the correct answers, and what comes out is... a program. That program can then be used on new data. This is **machine learning**.

### Traditional programming



Rules are typed in by a human.

### Machine learning



Rules (the model)

Rules are discovered from examples.

The same three ingredients — data, rules, answers — but the arrows point differently. In machine learning the rules are the *output*.

## BIG IDEA

Machine learning is the science (and art) of programming computers so that they can **learn from data**. Or, in Arthur Samuel's famous words, it is the field of study that gives computers the ability to learn **without being explicitly programmed**.

**THINK OF IT LIKE**

How did you learn to recognise your friend's handwriting? Nobody gave you rules ("the letter *a* has a loop that leans 12° to the left..."). You simply saw many samples with the name attached, and one day you could just tell. You were trained by examples. That is exactly the kind of learning we are about to give to a computer.

## The spam filter: rules versus learning

The slides use email spam as the first example, and it is a good one. Let us walk through it the two ways.

**The rule-writing way.** You notice that spam often contains "4U", "free", "credit card", "amazing". So you write a detector: *if the mail contains any of these words, mark it spam*. It works for a week. Then the spammers write "4 U" with a space, or "fr33". You add rules. They change again. Soon your program is a long, fragile list of hundreds of rules that nobody dares to touch, and it still misses half the spam.

**The learning way.** You collect thousands of emails that people have already marked as spam or "ham" (not spam). You let a learning algorithm count which words appear *unusually often* in spam compared with ham. The program itself finds that "4U", "free" and "fr33" are strong spam signals — and when spammers change their tricks next month, you just feed it the new examples and it re-learns. The program is **shorter, easier to maintain, and usually more accurate**.

**STOP AND THINK**

Meera wanted a program that prices used cars. Which of the two ways is she using at the start of the story? Why does it fail?

**OUR ANSWER**

She is writing rules by hand — traditional programming. It fails because the "rule" for a car's price depends on many things at once (brand, year, engine, mileage, distance travelled, whether it was a taxi...), and the way they combine is not something anyone can write down cleanly. The forty rules contradict each other because the real relationship is hidden in the data, not in her head.

## "It is very hard to say what makes a 2"

Geoffrey Hinton has a lovely slide with dozens of handwritten 2s. Every one is different — thick, thin, with a loop, without a loop, leaning left, leaning right. Yet you recognise each instantly. Now try to *write down* what makes a 2 a 2 and not a 7. You cannot. Your brain knows, but it cannot explain. This is the deepest reason machine learning exists:

**SPARK**

**Some tasks can only be defined by examples.** Recognising a face, a voice, a handwritten digit, a fraudulent transaction, a good chess move — humans do these well but cannot explain how. Where explanation fails, examples take over. Machine learning is how we hand a computer a pile of examples and say "figure it out".

## Where you already meet machine learning every day

The slides list a lot of applications; here they are grouped so they are easy to remember.

| Area                       | Examples you use or hear about                                                                                                  | What the machine learnt from                                                             |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Security & transactions    | Fraud detection in banking, email filtering, self-driving cars, dynamic pricing in travel, video surveillance, object detection | Past transactions labelled fraud/not, emails labelled spam/ham, hours of driving footage |
| Customer support           | Siri, Google Assistant, Alexa, Cortana, Bixby, chatbots                                                                         | Millions of spoken sentences with transcripts                                            |
| Recommendation engines     | Amazon, Flipkart, Netflix, IMDb, Goodreads, MakeMyTrip, Zomato                                                                  | What people like you bought, watched, rated                                              |
| Medicine, planning, gaming | Medical diagnosis, navigation and path finding, game playing, robot control                                                     | Labelled scans, maps and sensor logs, games played                                       |

Notice the third column. In every case there is a **pile of examples with the right answer attached**, or at least a pile of experience. No examples, no learning.

**IN ONE BREATH**

Traditional programming: you write the rules. Machine learning: you supply examples and the computer writes the rules. We use learning when the rules are too many, too fuzzy, or impossible for a human to state — which is most interesting problems.

## CHAPTER 2

# Task, Performance, Experience

*The one definition of learning you must be able to write in an exam — and how to apply it to any problem.*

**STORY**

Arjun listens to Meera's forty-rule disaster and smiles. "Forget the rules. Tell me three things instead. What exactly should the program *do*? How will we know it is doing it *well*? And what *past experience* can it learn from?" Meera writes the three questions on a sticky note. She will use that note for the rest of the course.

## Tom Mitchell's definition

The engineering definition of learning, from the course textbook (Mitchell, Chapter 1), sounds formal but is really Arjun's three questions:

**BIG IDEA**

A computer program is said to **learn** from experience  $E$  with respect to some task  $T$  and performance measure  $P$ , if its performance at task  $T$ , as measured by  $P$ , **improves with experience**  $E$ .

A well-defined learning problem is therefore a triple  $\langle T, P, E \rangle$ .

|                                  |                                                                                                                                        |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| $T$ — <b>Task</b>                | What the program must do. "Classify emails as spam or not." "Predict the price of a car."                                              |
| $P$ — <b>Performance measure</b> | A number that says how well it is doing. "Percentage of emails correctly classified."<br>"Average pricing error in rupees."            |
| $E$ — <b>Experience</b>          | The data or practice it learns from. "A database of emails with human labels."<br>"40,000 past listings with their final sale prices." |

**THINK OF IT LIKE**

A cricket coach and a young batter. The **task**: score runs against fast bowling. The **performance measure**: batting average in practice matches. The **experience**: a thousand balls faced in the nets. The batter is "learning" if, as the number of balls faced goes up, the average goes up too. Notice that all three have to be present: without a measure you cannot tell whether practice is helping; without practice there is nothing to learn from.

## The four examples from the slides, spelled out

| Problem     | $T$ (task)                                        | $P$ (performance)                                              | $E$ (experience)                                                                  |
|-------------|---------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Handwriting | Recognise hand-written words                      | % of words correctly classified                                | Database of human-labelled images of handwritten words                            |
| Spam        | Categorise email as spam or legitimate            | % of emails correctly classified                               | Database of emails, some with human labels                                        |
| Checkers    | Play checkers                                     | % of games won against opponents                               | Games played against itself                                                       |
| Driving     | Drive on a four-lane highway using vision sensors | Average distance travelled before an error (judged by a human) | A sequence of images and steering commands recorded while watching a human driver |

Look at the checkers row. The experience is *games played against itself* — there is no human teacher and no labelled dataset, just a lot of practice with a score at the end. That is a preview of **reinforcement learning** (Chapter 4). Look at the driving row: the machine watches a human and copies. That is **supervised learning** — learning from a teacher's answers.

### STOP AND THINK

Write  $\langle T, P, E \rangle$  for Meera's car-pricing problem at Sahi Deal. Be precise about  $P$ : what number would Arjun actually look at?

### OUR ANSWER

$T$ : predict the selling price of a used car from its description (brand, year, engine size, mileage, kilometres driven, whether it was a cab).  $P$ : the average absolute gap, in rupees, between the predicted price and the price the car actually sold for, measured on listings the model has never seen.  $E$ : the 40,000 past listings together with their actual sale prices. If the average gap shrinks as we feed in more past listings, the program is learning.

## How to write a good $P$

Students find  $T$  and  $E$  easy and  $P$  slippery. Three rules of thumb:

- **$P$  must be a number** you can compute from the program's answers, not a feeling. "Works well" is not a  $P$ . "Fraction of test emails classified correctly" is.
- **$P$  should be measured on new cases**, not the cases the program was trained on. A student who memorises last year's paper scores 100% on it and learns nothing. We care about the *unseen* exam.
- **$P$  should match what the business cares about**. For a cancer screen, "percentage of cancers caught" matters more than "percentage of all patients labelled correctly". (Session 5–6 makes this precise with precision, recall and ROC curves.)

**EXAM TIP**

A classic 2–3 mark question: "Define a well-posed learning problem and give  $T$ ,  $P$ ,  $E$  for [some application]." Write Mitchell's sentence first, then a three-line table. Make  $P$  a measurable quantity and  $E$  a concrete dataset or practice process — that is where marks are lost.

**IN ONE BREATH**

Learning = getting better at a task  $T$ , as measured by a number  $P$ , through experience  $E$ . Whenever you meet a new ML problem, write its  $\langle T, P, E \rangle$  before anything else.

## CHAPTER 3

# When Should You Use Machine Learning?

*Learning is powerful but not free. Four situations that call for it — and one that does not.*

## STORY

Excited, Meera proposes machine learning for everything: pricing, the payroll system, the GST calculator, the parking allocation. Arjun stops her at payroll. "Salary is basic pay plus allowances minus tax. We *know* the rule. Why would you make a computer guess something we can write in one line?"

## Four situations where learning wins

The slides give four situations. Each one is really the same situation in disguise: **the rule is not available to a human.**

| Situation                                              | Example                                                     | Why rules fail                                                                          |
|--------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>Human expertise does not exist</b>                  | Navigating on Mars                                          | Nobody has done it; there is no expert to interview.                                    |
| <b>Humans have the expertise but cannot explain it</b> | Recognising a face, a voice, a handwritten 2 (biometrics)   | The knowledge is in our brain but not in words.                                         |
| <b>The solution must be customised</b>                 | Personalised medicine, personalised recommendations         | One rulebook per person is impossible to write by hand.                                 |
| <b>The relationships are hidden in large data</b>      | Medical diagnosis from thousands of records, fraud patterns | Too many variables interacting for a human to encode; and new knowledge keeps arriving. |

## THINK OF IT LIKE

When do you look up a recipe, and when do you ask your grandmother to cook with you? If the dish has an exact recipe (boil rice: 1 cup rice, 2 cups water, 15 minutes), follow the recipe — that is traditional programming. If the dish is "make it taste like home", no recipe exists; you learn by watching and tasting many times — that is machine learning.

## And when learning is a waste

Learning is **not** always useful. Payroll, interest calculation, unit conversion, sorting a list — these have exact rules that we already know. A learnt model would be slower to build, harder to trust, and occasionally wrong, for a problem that has a perfect one-line answer. Use ML when the rule is unknown, not when it is inconvenient to type.

**BEFORE YOU START AN ML PROJECT, ASK**

1. **Is there a pattern to detect?** If the outcome is essentially random (tomorrow's lottery number), no amount of data will help.
2. **Can I solve it analytically?** If a formula or a lookup table already gives the answer, use that.
3. **Do I have data?** Enough examples, with the right answer attached (or at least enough experience to score). No data, no learning.

**STOP AND THINK**

For each of these, say ML or not-ML and why: (a) computing the EMI on a car loan; (b) deciding which of 40,000 listings a particular visitor is most likely to click; (c) checking whether a registration number is in the right format; (d) flagging listings that are probably scams.

**OUR ANSWER**

(a) Not ML — an exact formula exists. (b) ML — the "rule" for taste differs per visitor and is hidden in click histories. (c) Not ML — a format is a fixed rule (a pattern check). (d) ML — scammers' tricks are many, fuzzy and keep changing; learn them from examples of past scams.

**IN ONE BREATH**

Use learning when the rule is unknown, unexplainable, per-person, or buried in big data — and only if there is a pattern, no formula, and enough data. Otherwise, just write the code.

## CHAPTER 4

# The Three Families of Learning

*Supervised, unsupervised and reinforcement learning — sorted by one question: what kind of feedback does the learner get?*

## STORY

Meera now has three requests on her desk. Marketing wants to know *which customers are big spenders and which are bargain hunters* — but nobody has ever labelled a customer. Finance wants to predict *which loan applicants will default* — and they have years of yes/no history. And the robotics intern wants the warehouse trolley to *learn to park itself*. Arjun says: "Three different kinds of feedback. Three different families."

## Sort by feedback

Every learner gets some kind of feedback from its experience. The three families differ in **how much** and **when**:

| Family               | Feedback                                     | What the data looks like                                               | Goal                                                     |
|----------------------|----------------------------------------------|------------------------------------------------------------------------|----------------------------------------------------------|
| <b>Supervised</b>    | Full, immediate ("here is the right answer") | Pairs $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ — inputs with labels | Learn $f$ so that $f(x) \approx y$ on new $x$            |
| <b>Unsupervised</b>  | None ("no answers given")                    | Just $x_1, x_2, \dots, x_n$ — no labels                                | Find hidden structure: groups, compressed views          |
| <b>Reinforcement</b> | Delayed rewards and penalties                | An agent acting in an environment, getting a score now and then        | Learn a strategy that collects the most reward over time |

## THINK OF IT LIKE

**Supervised** is school with an answer key: for every practice question you are told the correct answer, and you adjust. **Unsupervised** is being handed a box of mixed Lego bricks and told "sort these" — nobody says how; you notice colours and shapes yourself. **Reinforcement** is training a puppy: no answer key, no explanation, just a treat when it sits and a firm "no" when it chews the sofa — and the puppy slowly works out what earns treats.



Left and middle: supervised learning with labels ( $y$ ). Right: unsupervised — the points come with no labels, yet three groups are plainly there.

## Supervised learning: classification and regression

In supervised learning we are given  $n$  examples  $(x_1, y_1), \dots, (x_n, y_n)$  and must learn a function  $f$  that predicts  $y$  from  $x$ . The **goal** is that previously unseen records are assigned a value as accurately as possible — it is *never* about the training examples themselves. The only question is: what kind of thing is  $y$ ?

### Classification — $y$ is a category

$y \in \{\text{benign, malignant}\}, \{\text{spam, ham}\}, \{\text{yes, no, maybe}\}$ . The tumour example on the slide:  $x$  = tumour size,  $y = 1$  if malignant,  $y = 0$  if benign. Learn the size beyond which we should say "malignant". Applications: image classification, face recognition, spam filters, document tagging, fraud detection.

### Regression — $y$ is a real number

$y \in \mathbb{R}$ : a price, a temperature, an area. The Arctic sea-ice example:  $x$  = year,  $y$  = ice extent in millions of km². Learn the trend line so that we can say something about a year not in the table. Meera's car price is a regression problem: the answer is a number of rupees.

### A LITTLE NOTATION, GENTLY

We write an input as  $x$  and the true output as  $y$ . A model is a function  $f$  (the slides also call it a *hypothesis*  $h$ ). Prediction is  $\hat{y} = f(x)$  — the hat means "our estimate". In classification  $y$  takes finitely many values; in regression  $y$  is continuous. When  $x$  has several attributes we write it as a vector  $\mathbf{x} = (x_1, x_2, \dots, x_d)$  — for the tumour,  $x_1$  = clump thickness,  $x_2$  = uniformity of cell size, and so on. Each attribute is one **dimension**; adding attributes is called **increasing the feature dimension**.

## Vocabulary you will see everywhere

The slides show three little tables. They teach the words that every dataset uses.

| S.No. | CGPA | Communication | Aptitude  | Programming | Job offered? |
|-------|------|---------------|-----------|-------------|--------------|
| 1     | 9.1  | Average       | Good      | Excellent   | Yes          |
| 2     | 8.4  | Good          | Good      | Good        | Yes          |
| 3     | 8.3  | Poor          | Average   | Average     | No           |
| 4     | 7.1  | Average       | Good      | Average     | No           |
| 5     | 8.2  | Good          | Excellent | Excellent   | No           |

Employability prediction. Each row is one **example / instance / record**. The first four columns are **features / attributes / predictors** (the  $x$ ). The shaded column is the **label / target / class** (the  $y$ ). Because  $y$  is Yes/No this is **classification**.

The used-car table on the slide (brand, year, engine capacity, mileage, distance travelled, cab?) with *Price* as the last column is the same picture with a numeric  $y$  — a **regression** table. The market-segmentation table (zip code, family income, visits per month, average spend) has *no* label column at all — that is what an **unsupervised** dataset looks like.

## The supervised workflow in one picture

Training: *images* → *features* → *training (with labels)* → *learned model*. Testing: *new image* → *features* → *learned model* → *prediction*. Two things to notice. First, we rarely feed raw data straight in — we first extract **features** (numbers that describe each example). Second, the model is built once on training data and then **reused** on new data; the labels are only needed during training.

## Unsupervised learning: structure without labels

Here we are given only  $x_1, \dots, x_n$  and must output the hidden structure behind them. The classic task is **clustering**: put similar records into groups so that points within a cluster are close together and points in different clusters are far apart (the slide says it formally: *intra-cluster distances minimised, inter-cluster distances maximised*). Marketing's "big spenders vs bargain hunters" is clustering — the groups are discovered, not given, and we might find two groups, or three, or four.

The second big task is **dimensionality reduction and visualisation**: squeeze a table with hundreds of columns down to two or three that keep most of the information (PCA, kernel PCA, LLE, t-SNE). Techniques named on the slide: k-means, hierarchical clustering, expectation-maximisation; PCA and friends. Applications: personalised recommendation, targeted marketing, grouping news articles, campaigning.

## Reinforcement learning: learning from consequences

An **agent** lives in an **environment**. It takes an **action**, the environment responds with a new situation and a **reward** (or penalty). Nobody tells the agent the right action; it must try things, see what happens, and gradually prefer actions that lead to more reward — *trial and error*. Rewards may come late: in chess you only

find out at the end whether move 12 was good. Reinforcement learning suits problems where **decisions are sequential and the goal is long-term**: games, robotics (the robot dog learning to move its legs), controlling a warehouse trolley.

## In between: semi-supervised learning

Labelling is expensive. Google Photos can cluster the faces in your library with no labels at all (unsupervised); then you name one photo "Amma", and it labels the whole cluster (a little supervision goes a long way). Data that is **partly labelled** — a few labels and a lot of unlabelled records — is handled by **semi-supervised** methods that combine the two families.

### STOP AND THINK

Classify each of Meera's three requests (customer segments, loan default, self-parking trolley) into a family, and say whether the supervised one is classification or regression.

### OUR ANSWER

Customer segments: no labels exist → **unsupervised** (clustering). Loan default: years of yes/no history → **supervised classification** ( $y \in \{\text{default, no default}\}$ ). Self-parking trolley: actions, consequences, delayed reward (parked well or bumped) → **reinforcement learning**.

### EXAM TIP

"Compare supervised, unsupervised and reinforcement learning" is a standard question. Score it with a three-row table: *feedback* (labels / none / delayed rewards), *data* (pairs  $(x, y)$  / only  $x$  / agent–environment interaction), *goal* (predict  $y$  / find structure / maximise cumulative reward), and one *example* each. Then add the classification-vs-regression split inside supervised.

### IN ONE BREATH

Supervised: learn from answers (classification if the answer is a category, regression if it is a number). Unsupervised: find structure with no answers (clustering, dimensionality reduction). Reinforcement: learn from delayed rewards by acting. Semi-supervised: a few labels, many unlabelled records.

## CHAPTER 5

# Two More Ways to Slice Learning

*How much data the learner swallows at a time, and whether it keeps the data or keeps a rule.*

## STORY

"One more thing," says Arjun. "New listings arrive every minute. Do you retrain your price model from scratch every night on all 40,000 rows, or nudge it a little with each new car? And when a new car arrives — do you look up similar cars, or do you have a formula?" Meera realises the same problem can be attacked in quite different styles.

## Slice 1 — how the training data is used

### Batch learning

Use **all** the available data at once for each training pass. Simple and stable, but every update looks at everything, so it is slow for big data and cannot easily absorb new data without retraining.

### Mini-batch learning

Use a **subset** (say 32 or 256 rows) at a time. A compromise: faster updates than batch, less noisy than one-at-a-time. This is the default in deep learning.

### Online (incremental) learning

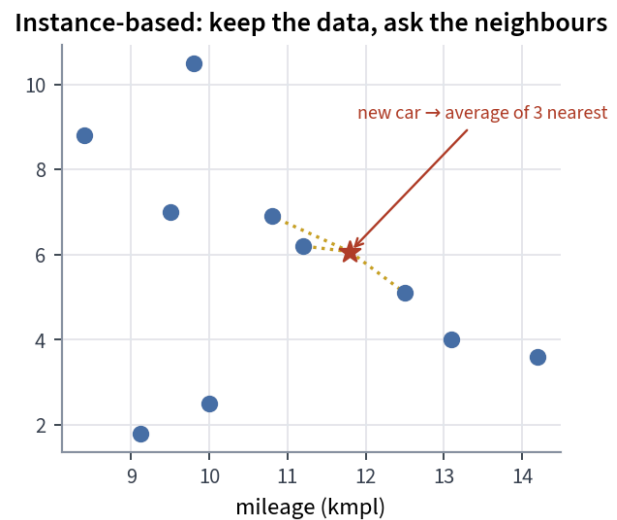
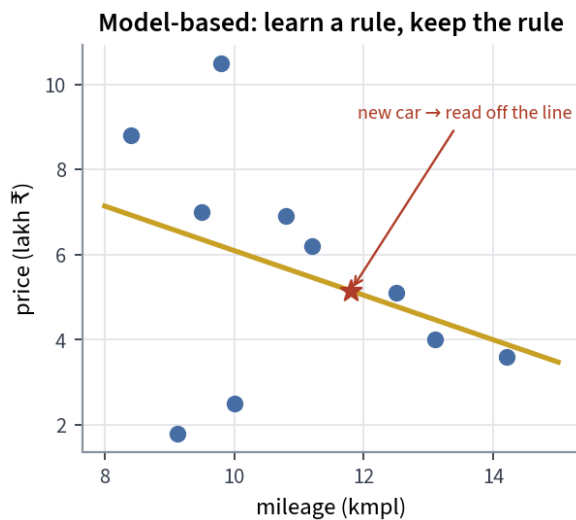
Use **one** record at a time and update immediately. Perfect when data streams in continuously (stock ticks, new listings) or does not fit in memory. Cheap per step, but each step is noisy.

## THINK OF IT LIKE

Preparing for an exam. **Batch**: read the entire textbook, then revise your understanding once. **Mini-batch**: read a chapter, update your notes, next chapter. **Online**: update your notes after every single page. All three get you there; they differ in how often you stop to adjust and how steady each adjustment is. (Session 3–4 turns this into batch, mini-batch and stochastic gradient descent, with formulas.)

## Slice 2 — instance-based versus model-based

There are two ways to generalise from examples.



Same data, two philosophies. Model-based: fit a line once, then throw the data away and use the line. Instance-based: keep every point; to price a new car, find its nearest neighbours and average them.

### Instance-based learning

#### Compare new data points to known data points.

The learner memorises the training examples. To answer a new query, it finds the most similar stored examples and copies (or averages) their answers. Example:  $k$ -nearest neighbours (Module 6). No real "training" happens; all the work is at prediction time.

### Model-based learning

#### Detect patterns in the training data and build a predictive model.

The learner boils the examples down to a compact rule — a line, a tree, a set of weights — and uses the rule for new queries. Example: linear regression (Module 3), logistic regression (Module 4), decision trees (Module 5). Training is the hard part; prediction is instant.

#### THINK OF IT LIKE

Two ways to price a flat. Instance-based: look at the last five sales in the same building and quote the average — you need the whole sales register at hand every time. Model-based: from all past sales, learn "₹9,500 per sq ft plus ₹4 lakh per parking slot" and quote from the formula — the register can be put away.

#### STOP AND THINK

Which style would you choose for Sahi Deal's price engine if (a) the catalogue is small and prices depend on rare combinations, or (b) there are millions of listings and you must quote in one millisecond?

#### OUR ANSWER

(a) Instance-based: with few records and quirky combinations, "find the most similar cars" is honest and needs no assumptions. (b) Model-based: a compact formula answers in a millisecond and does not need to search millions of rows at query time.

## Tools of the trade

The slides list open-source tools. You need to recognise the names, not memorise the table.

| Tool                              | Language            | Known for                                                                                                                          |
|-----------------------------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>scikit-learn</b>               | Python              | Classification, regression, clustering, preprocessing, model selection, dimensionality reduction — <i>the tool for this course</i> |
| PyTorch, TensorFlow, Keras        | Python (C++ core)   | Neural networks; automatic gradients (autograd), optimisation modules                                                              |
| Weka                              | Java                | Data preparation, classification, regression, clustering, visualisation, association rules — GUI-driven                            |
| Apache Mahout, Shogun, Accord.NET | Java/Scala, C++, C# | Distributed linear algebra and recommenders; SVMs; .NET ecosystem                                                                  |
| Google Colab                      | Cloud               | Free notebooks with PyTorch/TensorFlow/Keras/OpenCV preinstalled — where the labs run                                              |

### IN ONE BREATH

Batch / mini-batch / online: all the data, a chunk, or one record per update. Instance-based keeps the examples and asks the neighbours; model-based compresses the examples into a rule. scikit-learn is the workhorse for this course.

## CHAPTER 6

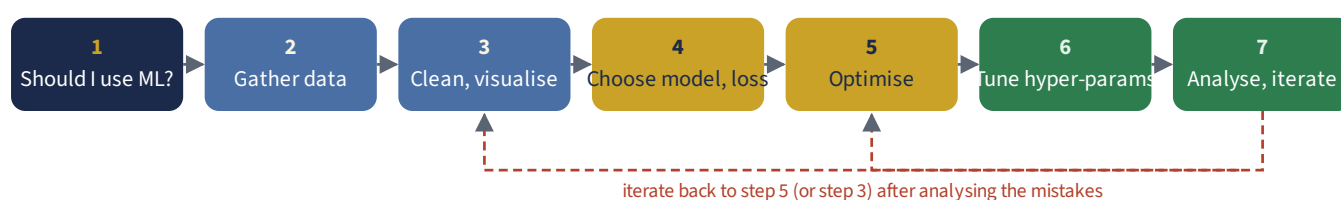
# The Machine Learning Workflow

*Seven steps that every ML project goes through — walked end to end on the car-price problem.*

## STORY

Meera finally starts the real project. Arjun pins a checklist above her desk with seven steps and one rule: "Nobody touches step 5 before steps 1 to 4 are done. Most failed projects are failures of step 1 or step 3, not of the algorithm."

## The seven steps



The ML workflow from the slides. Steps 1–3 are about the problem and the data; 4–5 are the "learning"; 6–7 are about making it good.

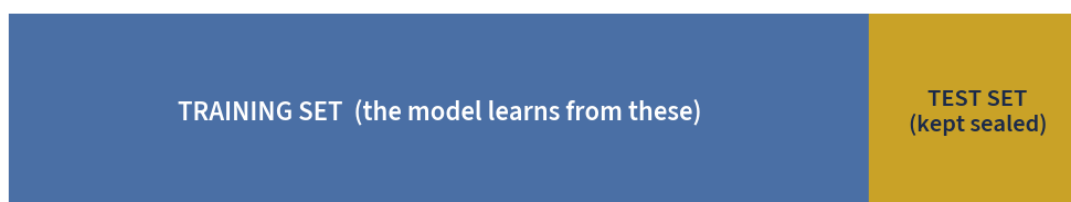
- 1. Should I use ML on this problem?** Is there a pattern? Can I solve it analytically? Do I have data? (Chapter 3.)
- 2. Gather and organise data.** Surveys, logs, past purchases, sensors. Decide what a row is and what the label is.
- 3. Preprocess, clean, visualise.** Fix missing values, outliers, duplicates; scale features; look at plots. (All of Session 2.)
- 4. Choose a model, a loss, regularisation.** Decide the *form* of the rule (a line? a tree?), the *score* that says how wrong it is (the loss or objective function), and any *penalty* for over-complicated rules.
- 5. Optimise.** Find the parameter values that make the loss as small as possible (closed-form formulas or gradient descent — Session 3–4).
- 6. Hyper-parameter search.** Settings that the optimiser does not learn (learning rate, tree depth, regularisation strength) are chosen by trying values on a validation set.
- 7. Analyse performance and mistakes; iterate.** Look at where the model is wrong, then go back to step 5 (re-optimize) or step 3 (better features or cleaner data).

## Walking it through: car price from mileage

The slides do exactly this walk-through; here it is with Meera's numbers.

| Step on the slide                          | What Meera actually does                                                                                                                                   |
|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Define the objective                       | $T$ : predict price from mileage. $P$ : average error on unseen cars.                                                                                      |
| Data gathering                             | Survey of recent sellers + the past purchase records: pairs (mileage, price).                                                                              |
| Data preprocessing                         | Split into a <b>training set</b> and a <b>test set</b> ; decide how to represent the input (mileage in kmpl as a number) and the output (price in lakh ₹). |
| Exploratory data analysis                  | Scatter-plot price against mileage. Does it look roughly like a line? Any crazy points?                                                                    |
| Choose the form of the model               | Linear regression: $\text{price} = \theta_0 + \theta_1 \cdot \text{mileage}$ . Two numbers to learn.                                                       |
| Performance evaluation: objective function | Sum of squared errors between predicted and actual prices over the training set.                                                                           |
| Optimisation                               | Set $\theta_0, \theta_1$ to whatever makes the objective smallest.                                                                                         |
| Evaluate on the test set: generalisation   | Compute the error on the sealed test cars. This — not the training error — is the number Arjun cares about.                                                |

## The one idea that runs the whole course: generalisation



Before learning anything, seal away a test set. The model never sees it during training.

### BIG IDEA

The purpose of a model is to be right on data it has **never seen**. Performance on the training set is not evidence of learning; performance on the held-out test set is. The ability to do well on new data is called **generalisation**.

### THINK OF IT LIKE

Two students prepare for an exam from the same question bank. One memorises the answers to every question; the other understands the methods. On the question bank both score 100%. On the real exam, with new questions, only the second student does well. The real exam is the test set. Memorising is what a model does when it fits the training data too closely — we will call it **overfitting**.

## Three vocabularies for the same thing

Different books use different words for the same objects. The table saves you confusion later:

| Statistics says                 | ML says                              | Database says         |
|---------------------------------|--------------------------------------|-----------------------|
| observation, case               | example, instance, sample, point     | record, row, tuple    |
| independent variable, predictor | feature, attribute, input, dimension | column, field         |
| dependent variable, response    | label, target, output, class         | the column we predict |
| coefficients                    | parameters, weights                  | —                     |

### EXAM TIP

"Explain the steps of a typical ML workflow with an example" is a favourite. List the seven steps in order, then illustrate each with the same running example (car price, house price, spam). Mention the train/test split explicitly and say *why*: to estimate performance on unseen data (generalisation).

### IN ONE BREATH

Decide whether ML fits → gather → clean and look → choose model + loss → optimise → tune → analyse and iterate. Seal a test set before you start; only the test error tells you whether the model generalises.

## CHAPTER 7

# What Can Go Wrong (a Preview)

*The issues in machine learning that the rest of the course is built to solve.*

## STORY

Meera's first model prices every car in the training file almost perfectly. She is thrilled. Arjun tries it on last week's new listings and it is off by lakhs. "It memorised," he says. "And by the way — did you notice that nine out of ten cars in your file are hatchbacks? Try it on an SUV." Meera has just met the two classic issues of ML in one afternoon.

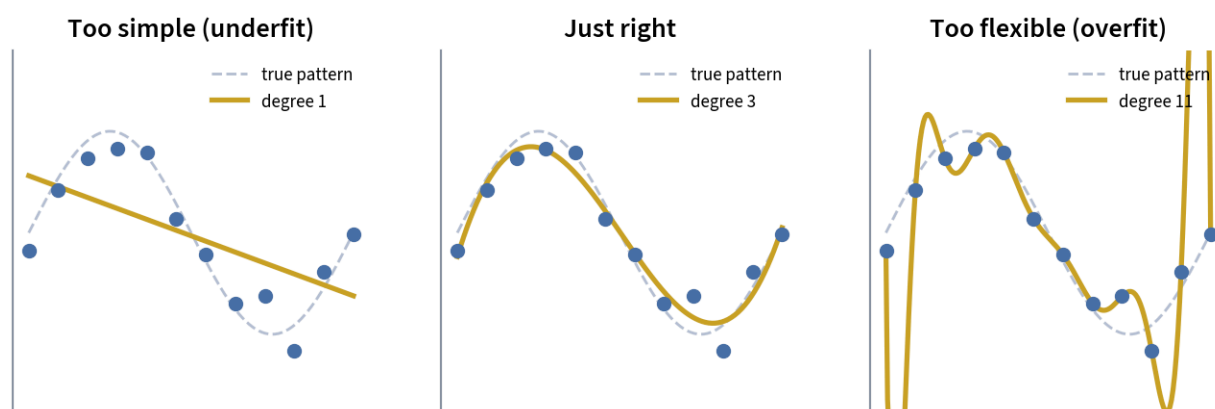
## Issue 1 — Not enough data

Learning needs examples. With ten cars, any pattern you find is probably an accident. With ten thousand, real patterns stand out from noise. The slides put it as a trade-off between the **algorithm** and the **data readiness**: a fancy model with little data is worse than a simple model with plenty. The remedy is usually boring — collect more, or choose a simpler model that needs less.

## Issue 2 — Non-representative data

If the training set is mostly hatchbacks, the model learns hatchback prices. Show it an SUV and it guesses like a hatchback. Even a large sample is useless if it was collected in a biased way (**sampling bias**); a small sample adds randomness on top (**sampling noise**). The training data must be **representative** of the cases we want to generalise to. Session 2 covers sampling properly.

## Issue 3 — Overfitting and underfitting



The same twelve points fitted three ways. Left: a straight line misses the wave. Middle: a modest curve follows the true pattern. Right: a very flexible curve passes through every point — including the noise — and would be wild between and beyond them.

### Underfitting

The model is **too simple** to capture the pattern. It does badly even on the training data. Cure: a richer model, better features.

### Overfitting

The model is **too flexible**: it fits the training data — noise and all — and does badly on new data. Cure: more data, a simpler model, or a penalty on complexity (regularisation).

#### THINK OF IT LIKE

A tailor with one customer. Underfitting: he stitches a one-size sack — fits nobody well. Overfitting: he stitches a suit so exactly to this customer's posture on Tuesday afternoon that it pinches on Wednesday. The right fit follows the shape but leaves room to move.

## Issue 4 — Validation and testing

If you tune your model by looking at the test set, the test set stops being a fair exam — you have peeked at the paper. So in practice we use **three** sets: train (to fit), validation (to choose settings), test (used once, at the very end). Session 2 sets this up; Sessions 3–6 use it constantly.

#### CAREFUL

The slide says these issues "will be discussed in later modules". For the exam that means: you should be able to *name and one-line-explain* them now, and *work with them numerically* after Sessions 3–6 (bias-variance, learning curves, regularisation, confusion matrices).

#### IN ONE BREATH

Four things go wrong: too little data, unrepresentative data, a model that is too simple (underfit) or too flexible (overfit), and testing on data you tuned on. Everything after Session 1 is a tool for one of these.

## CHAPTER 8

# Self-Check and the Road Ahead

*Twelve quick questions to test yourself, with answers, and a map of how Session 1 connects to the rest of the course.*

## Twelve questions (cover the right column)

| #  | Question                                                                | Answer                                                                                                                                                                    |
|----|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | In traditional programming, what goes in and what comes out? And in ML? | Traditional: data + rules in, answers out. ML: data + answers in, rules (a model) out.                                                                                    |
| 2  | State Mitchell's definition of learning.                                | A program learns from experience $E$ w.r.t. task $T$ and performance measure $P$ if its performance at $T$ , measured by $P$ , improves with $E$ .                        |
| 3  | Give $\langle T, P, E \rangle$ for a spam filter.                       | $T$ : classify emails as spam/legitimate; $P$ : % correctly classified; $E$ : a database of emails with human labels.                                                     |
| 4  | Name two situations where ML is appropriate and one where it is not.    | Appropriate: humans cannot explain their expertise (face recognition); the solution must be customised (personalised medicine). Not: payroll — the rule is known exactly. |
| 5  | What distinguishes supervised from unsupervised learning?               | Supervised data has labels $y$ and we learn to predict them; unsupervised data has only $x$ and we look for hidden structure.                                             |
| 6  | Classification vs regression — the one-line test?                       | Is $y$ categorical (classification) or a real number (regression)?                                                                                                        |
| 7  | Name three unsupervised techniques.                                     | k-means, hierarchical clustering, expectation-maximisation; PCA, t-SNE, LLE for dimensionality reduction.                                                                 |
| 8  | What is the feedback signal in reinforcement learning?                  | Delayed rewards and penalties from the environment after the agent's actions.                                                                                             |
| 9  | Batch vs online learning?                                               | Batch uses all data for each update; online updates one instance at a time (mini-batch: a subset).                                                                        |
| 10 | Instance-based vs model-based?                                          | Instance-based compares new points with stored points (kNN); model-based learns a compact predictive rule (regression, trees).                                            |
| 11 | Why split data into training and test sets?                             | To measure generalisation — performance on unseen data — which is the real goal.                                                                                          |
| 12 | Define overfitting in one sentence.                                     | The model fits the training data (including its noise) so closely that it performs poorly on new data.                                                                    |

## How Session 1 connects to everything else

| Idea from this book                           | Where it grows up                                                                 |
|-----------------------------------------------|-----------------------------------------------------------------------------------|
| Step 3 of the workflow (clean, scale, sample) | Session 2 — ML workflow, data preprocessing, feature engineering                  |
| Regression; loss; optimisation; overfitting   | Sessions 3–4 — linear regression, gradient descent, bias–variance, regularisation |
| Classification; the right $P$                 | Sessions 5–6 — logistic regression, confusion matrix, precision/recall, ROC       |
| Model-based learning with rules you can read  | Sessions 7–8 — decision trees, pruning                                            |
| Instance-based learning                       | Module 6 — k-nearest neighbours                                                   |
| Unsupervised learning                         | Module 10 — k-means, mixture models                                               |

### SPARK — THE SENTENCE TO CARRY WITH YOU

Every learning algorithm you will meet is the same three-beat rhythm: **guess** (a model with adjustable parameters), **measure** (a performance number), **adjust** (change the parameters to improve the number). Linear regression, logistic regression, neural networks, trees — all of them. Once you see the rhythm, the course stops being a list of algorithms and becomes one idea wearing different clothes.

### STORY — TO BE CONTINUED

Meera closes her laptop with a plan: gather the listings, clean them, plot them, then fit her first line. Tomorrow she will discover that the data is much messier than the slides make it look — and that is Book 2.